

0: C-Bootcamp

TUTORIUM 24 - BuS 2024

Aufgabe 1 – Arrays in C: Grundlagen

- zusammenhängender Block im Speicher mit fixer Länge

```
int a[4];  
a = {0,1,2,3};
```

- kein String-Datentyp – stattdessen als char-Array dargestellt

```
char s[11] = "helloworld"; //10 Zeichen + \0  
char t[] = "helloworld";
```

- Zugriff auf Arrayelemente durch Index

```
a[0] = 1; // -> {1,1,2,3}
```

Aufgabe 1 – Arrays in C: Vigenère-Chiffre

- Klartext wird durch Codewort verschlüsselt (Buchstabe für Buchstabe)
- Vorarbeit:
 - Nachricht und Code zu ALLCAPS
 - Leerzeichen entfernen
 - Codewort muss so oft wiederholt werden, dass jedes Zeichen aus dem Klartext abgedeckt ist
 - Indexe werden zugewiesen: A-Z auf 0-25 und 0-9 auf 26-35 -> Modulo beachten!

Code: BUS2022

Ciphertext : E2WK4JLFHBKEDDWY9K29DVYAK4DLXY954F

Codeword : BUS2022BUS2022BUS2022BUS2022BUS202

Plaintext : DIESERTEXTSOLLVERSCHLUESSELTWERDEN

Plaintext : Dieser Text soll verschlüsselt werden.

Aufgabe 1 – Arrays in C: Aufgabenstellung

Programm schreiben, dass...

- nach einem Codewort fragt (max. 10 Zeichen)
- nach einem Text fragt (max. 100 Zeichen)
- fragt, ob es chiffrieren oder dechiffrieren soll
- auf den Angaben basierend, die Wünsche des Users umsetzt und das Ergebnis ausgibt

Aufgabe 2a & b – siehe Code-Musterlösung

Bei Fragen gerne eine Mail schreiben oder nach dem nächsten Tutorium zu mir kommen

Zu b)

Vorteile:

- Neue Liste ist kleiner
- Bei der Modifizierung von Daten muss nur eine der Listen bearbeitet werden (sofern unser Sortierkriterium nicht verändert wird)

Aufgabe 2c

Welche der Funktionen (`enqueue()`, `dequeue()`, `is_empty()` & `get_student()`) müssten überarbeitet werden, um eine neue Anforderung im System umzusetzen (Hörsäle mit Bezeichnung und GPS-Koordinaten statt Studenten und Matrikelnummern)?

- Um die Anforderung umzusetzen, muss ein neuer struct angelegt werden
- Daher müssen alle Funktionen, die den alten struct *stud_type* benutzen, angepasst werden
- In unserem Beispiel heißt das konkret, dass nur *is_empty()* so bleiben kann, alle anderen Funktionen müssen angepasst werden
- Ohne generische Typen, die in C nur in speziellen Libraries vorhanden sind, ist das Erstellen von solchen Listen sehr aufwändig, ebenso wie eine Modifikation dieser

Aufgabe 2d - überspringen

Aufgabe 2e

Was macht dieses Makro? Was gibt es zurück?

```
#define position(typ , el) (( unsigned long) (&(( typ *)0)->el))
```

```
//Beispielnutzung: position(stud_type, next_student)
```

- Siehe *offsetof*
- Input: structtype und ein Element aus diesem struct
- In diesem Beispiel: *stud_type* und *next_student*
- Rückgabe: Abstand im Speicher von *next_student* zum Beginn vom *stud_type*
- Offset ist architekturenspezifisch
 - Könnte zum Beispiel 44 sein:
 - 4 (sizeof(int)) + 20 (char) + 20 (char)

Aufgabe 2f

Was ist falsch an dieser Funktion? Warum ist sie in den meisten Fällen nutzlos (unabhängig davon, was sie implementiert)?

```
int* what_am_i_doing_wrong(void) {  
    int x;  
    /* Something meaningful happens  
    * to the variable here ,  
    * e.g. x = 42; */  
    return &x;  
}
```

- x ist eine lokale Variable, sie existiert also nur zur Laufzeit der Methode
- Rückgabe der Adresse dieser lokalen Variable ist also sinnfrei, an der Adresse könnte inzwischen etwas anderes stehen
- Generell können keine Stackadressen zurückgegeben werden